Metaheurísticas

Aula 4 — Busca Tabu: Memória, Aspiração e Diversificação

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, o aluno deve ser capaz de:

1. Explicar a diferença filosófica entre o mecanismo de escape do SA (probabilístico) e o da Busca Tabu (determinístico por memória);
2. Implementar a lista tabu de curto prazo com tamanho variável e política de envelhecimento FIFO;
3. Projetar critérios de aspiração adequados ao problema;
4. Implementar estratégias de memória de longo prazo para intensificação e diversificação;
5. Aplicar Busca Tabu ao TSP e ao Problema da Mochila com análise crítica dos resultados;
6. Comparar SA e Busca Tabu com metodologia estatística rigorosa.

1 Motivação: da probabilidade à memória

Na Aula 3, vimos que o SA escapa de ótimos locais aceitando pioras com probabilidade $e^{-\Delta f/T}$. Essa abordagem é elegante e relativamente fácil de implementar, mas tem uma limitação fundamental: ela é **amnemésica**. O algoritmo não sabe quais regiões já foram exploradas e pode, portanto, revisitar o mesmo ótimo local múltiplas vezes — desperdiçando avaliações de f.

No TSP com $n = 30$ cidades, por exemplo, o SA com temperatura baixa frequentemente oscila entre dois ou três tours de qualidade semelhante, trocando arcos de ida e volta sem progredir. Cada oscilação consome avaliações do orçamento sem produzir informação nova.

A **Busca Tabu** (Tabu Search, TS), proposta por Fred Glover em 1986 e formalizada em 1989–1990, adota uma filosofia radicalmente diferente: em vez de usar probabilidade para escapar de ótimos locais, usa **memória explícita** para proibir o retorno a soluções recentemente visitadas. A palavra "tabu" — do polinésio, via inglês — designa algo proibido por convenção social. Na metaheurística, um movimento é tabu se a memória de curto prazo registra que ele reverteria um passo recente.

Essa mudança de perspectiva tem consequências profundas: a Busca Tabu é **determinística dado um estado inicial**, oferece controle preciso sobre o balanço exploração/explotação, e é particularmente eficaz em problemas combinatórios com vizinhanças bem definidas — exatamente o caso do TSP e do Knapsack.

2 Estrutura da Busca Tabu

2.1 Visão geral do framework

A Busca Tabu não é um único algoritmo, mas um **framework** que combina quatro componentes:

1. **Geração de vizinhança**: define quais movimentos são possíveis a partir da solução corrente.
2. **Memória de curto prazo** (lista tabu): proíbe movimentos que revertam passos recentes.
3. **Critério de aspiração**: permite ignorar a proibição tabu em circunstâncias excepcionais.
4. **Memória de longo prazo**: guia estratégias de intensificação e diversificação ao longo da busca.

A interação entre esses componentes é o que distingue implementações simples de implementações de alto desempenho.

2.2 O algoritmo núcleo

Algoritmo — Busca Tabu — Núcleo

```
BuscaTabu(f, Vizinhanca, tamanho_tabu, max_iter):
  s      ← solucao_inicial()        // pode ser aleatória ou heurística
  s*     ← s                        // melhor solução global
  LT     ← fila_vazia()             // lista tabu (FIFO)
  k      ← 0

  enquanto k < max_iter:
    candidatos ← {}
    para cada movimento m em Movimentos(s):
      s' ← aplica(s, m)
      se m ∉ LT  ou  criterio_aspiracao(s', s*):
        candidatos ← candidatos ∪ {(s', m)}

    // escolhe o melhor candidato (mesmo se piorar f)
    (s_melhor, m_melhor) ← argmin_f(candidatos)
    s ← s_melhor

    // atualiza lista tabu (FIFO)
    LT.push(inverso(m_melhor))
    se |LT| > tamanho_tabu: LT.pop()

    // atualiza melhor global
    se f(s) < f(s*): s* ← s
    k ← k + 1

  retorna s*
```

Três aspectos merecem atenção imediata:

1. O algoritmo sempre se move para o melhor candidato disponível — mesmo que esse candidato seja pior que a solução corrente. Isso contrasta com o HC (que para se não há melhora) e com o SA (que aceita pioras com probabilidade). Na BT, nunca há estagnação local forçada — o algoritmo **sempre** avança.

2. A lista tabu armazena o inverso do movimento, não a solução visitada. No TSP com operador 2-opt, se o movimento inseriu os arcos (i, j) e (k, l), o tabu registrado proíbe reinserir os arcos removidos (i, k) e (j, l). Isso impede desfazer o passo anterior sem necessitar armazenar tours completos.

3. O melhor global s^* é mantido separadamente. Como o algoritmo aceita pioras, a solução corrente s pode ser pior que s^*. Ao final, retorna-se s^*.

3 Memória de Curto Prazo: a Lista Tabu

3.1 Representação e estrutura

A lista tabu é uma fila FIFO (**First In, First Out**) de tamanho fixo θ (o **tenure** tabu). Quando um movimento m é executado, seu inverso $\overline{m}$ é inserido na lista. Após θ iterações, $\overline{m}$ é automaticamente removido — o movimento "expira" e pode ser executado novamente.

Definição — Lista Tabu e Tenure
Seja M o conjunto de todos os movimentos possíveis e $\overline{m}$ o inverso de $m \in M$. A **lista tabu** é o conjunto:

$$LT_k = \{\overline{m}_{k-\theta+1}, \overline{m}_{k-\theta+2}, ..., \overline{m}_k\}$$

onde m_i é o movimento executado na iteração i e θ é o **tenure** (tamanho da lista). Um movimento m' na iteração $k+1$ é **tabu** se $m' \in LT_k$.

O tenure θ é o parâmetro mais importante da lista tabu:

- **θ pequeno (2–5)**: proibições curtas — o algoritmo pode revisitar soluções rapidamente; risco de ciclagem.
- **θ grande (20–50)**: proibições longas — exploração mais ampla, mas pode proibir movimentos prometedores.
- **θ adaptativo**: varia dinamicamente com base na qualidade das soluções encontradas.

3.2 Tenure adaptativo

Uma estratégia eficaz é variar o tenure de forma aleatória a cada iteração:

$$\theta_k = \theta_{\min} + \lfloor (\theta_{\max} - \theta_{\min}) \cdot \text{Uniforme}(0,1) \rfloor$$

Alternativamente, aumentar θ quando a busca estagna (poucas melhorias consecutivas) e reduzi-lo quando há progresso constante — uma forma de auto-regulação do trade-off exploração/explotação.

3.3 O problema da ciclagem

Sem a lista tabu, o HC e o SA podem revisitar soluções. Com a lista tabu, a ciclagem de curto prazo é evitada — mas ciclos longos (período $> \theta$) ainda são possíveis. Para detectar ciclagem de longo prazo, algumas implementações usam uma tabela hash de estados visitados, mas isso é caro em memória para problemas grandes.

4 Critério de Aspiração

A lista tabu é uma heurística: proibições que evitam ciclagem de curto prazo. Mas ela pode proibir movimentos que levariam a soluções excepcionalmente boas. O **critério de aspiração** é a válvula de escape: permite ignorar a proibição tabu quando uma condição especial é satisfeita.

Definição — Critério de Aspiração
Uma função $A : \mathcal{F} \to \{\text{verdadeiro}, \text{falso}\}$ define o **critério de aspiração**: se $A(s')$ é verdadeiro para um movimento tabu, a proibição é levantada e s' pode ser aceito.

4.1 Critério de aspiração por qualidade (padrão)

O critério mais usado — e suficiente para a maioria dos problemas:

$$A(s') = [f(s') < f(s^*)]$$

Se o vizinho tabu produz uma solução **melhor que o melhor global conhecido**, a proibição é ignorada. A lógica é clara: se encontramos algo genuinamente novo e melhor, não faz sentido recusar — a proibição foi criada para evitar retrocesso, não progresso.

4.2 Critério de aspiração por origem

Uma variante mais sofisticada levanta a proibição se o movimento leva a uma região do espaço que não foi visitada nas últimas Ω iterações (rastreado por memória de frequência). Útil quando o critério por qualidade é raramente ativado — sinal de que a busca está estagnada numa região de qualidade homogênea.

4.3 Critério de aspiração por influência

Usado em problemas com estrutura especial: levanta a proibição se o movimento altera significativamente a estrutura da solução. No TSP, isso pode corresponder a um movimento que reconfigura uma região geográfica inteira do tour, em vez de apenas permutar dois arcos próximos.

⚠️ **Atenção**
Sem critério de aspiração, a Busca Tabu pode ficar presa mesmo tendo identificado um vizinho tabu que é claramente o melhor movimento disponível. O critério por qualidade ($f(s') < f(s^*)$) deve ser sempre implementado como mínimo.

5 Memória de Longo Prazo: Intensificação e Diversificação

A memória de curto prazo (lista tabu) governa a dinâmica local da busca. A **memória de longo prazo** governa a estratégia global — onde concentrar esforço (**intensificação**) e como forçar exploração de regiões novas (**diversificação**).

5.1 Memória de frequência

A estrutura fundamental é a **matriz de frequência** Φ, que conta quantas vezes cada atributo apareceu nas soluções visitadas. A forma concreta de Φ depende da representação do problema:

- **TSP**: $\Phi[i][j]$ conta quantas vezes o arco (i, j) apareceu nos tours visitados ao longo da busca.
- **Knapsack**: $\Phi[i][v]$ conta quantas vezes o item i teve valor $x_i = v$ (0 ou 1) nas soluções visitadas.

A frequência permite duas estratégias complementares:

Intensificação: ao identificar atributos de alta qualidade (e.g., o arco (i, j) aparece em 80% das melhores soluções encontradas), reinicia a busca fixando esses atributos — "exploração profunda" de uma região promissora.

Diversificação: quando a busca estagna, penaliza soluções com alta frequência adicionando um termo à função objetivo:

$$f_{\text{div}(s)} = f(s) + \lambda \sum_{\text{atributos}} \Phi[\text{atributo}(s)]$$

Isso força o algoritmo para regiões pouco visitadas. O coeficiente λ controla a intensidade da diversificação e deve ser calibrado para que a penalidade represente tipicamente 10–20% do valor de f.

5.2 Estratégia de intensificação-diversificação

A alternância entre as duas fases pode ser controlada por um contador de estagnação:

Algoritmo — Busca Tabu com Intensificação e Diversificação

```
BuscaTabuCompleta(f, Vizinhanca, θ, max_iter, freq_intens, freq_div):
  s, s*, LT, Φ, Elite ← inicializar()
  contador_estag ← 0

  enquanto k < max_iter:
    (s, m) ← melhor_candidato_nao_tabu(s, LT, s*, f)
    atualizar_LT(LT, m, θ)
    atualizar_frequencia(Φ, s)

    se f(s) < f(s*):
      s* ← s
      salvar_elite(Elite, s)
      contador_estag ← 0
    senão:
      contador_estag ← contador_estag + 1

    // diversificação: força saída de regiões muito visitadas
    se contador_estag = freq_div:
      s ← perturbacao_baseada_em_frequencia(Φ)
      LT ← fila_vazia()      // reseta lista tabu
      contador_estag ← 0

    // intensificação: refinamento profundo na melhor região
    se k mod freq_intens = 0  e  k > 0:
      s ← reinicio_baseado_em_elite(Elite)
      θ ← θ_min              // tenure pequeno para refinar

  retorna s*
```

5.3 Memória baseada em elite

Além da frequência, pode-se manter um **conjunto de soluções elite** — as E melhores soluções encontradas durante toda a busca. Essas soluções servem como pontos de partida para a fase de intensificação e como "âncoras" para identificar atributos comuns a boas soluções.

No TSP, a análise dos arcos comuns às soluções elite pode revelar "arestas obrigatórias" — arcos que provavelmente pertencem ao ótimo global. No Knapsack, revela quais itens tendem a estar presentes nas melhores soluções viáveis.

6 Aplicação: Busca Tabu no TSP e no Knapsack

6.1 Mapeamento TSP → Busca Tabu

O TSP com operador 2-opt mapeia naturalmente para o framework:

Solução: permutação $\pi = (\pi_1, ..., \pi_n)$ das n cidades.

Movimentos: pares de arcos $(i, i+1)$ e $(j, j+1)$ a remover, com $i < j$. Há $\frac{n(n-3)}{2}$ movimentos possíveis.

Inverso do movimento: o 2-opt é seu próprio inverso — reaplicar o mesmo par de índices (i, j) desfaz o movimento. A lista tabu registra portanto os pares de índices (i, j) diretamente.

Tenure: $\theta \approx \sqrt{n}$ como ponto de partida (Glover & Laguna, 1997). Para $n = 30$: $\theta \approx 5$–6.

Frequência: $\Phi[i][j]$ conta ocorrências do arco (i, j) nas soluções visitadas.

Inicialização: o algoritmo do vizinho mais próximo (*nearest neighbor*) produz um tour inicial de qualidade razoável em $O(n^2)$, colocando a busca numa região melhor que um ponto aleatório.

6.2 Mapeamento Knapsack → Busca Tabu

Fio Condutor — TSP e Knapsack

O Problema da Mochila com $n = 20$ itens (instância da Aula 1) também mapeia diretamente:

Solução: vetor binário $\boldsymbol{x} = (x_1, ..., x_n) \in \{0, 1\}^n$.

Movimentos: flip de um único bit $x_i : 0 \leftrightarrow 1$. Há n movimentos possíveis por iteração.

Inverso do movimento: flip do mesmo bit i — portanto a lista tabu registra apenas os índices i recentemente alterados.

Tenure: $\theta \approx \sqrt{n} \approx 4$–5 para $n = 20$.

Viabilidade: flips que tornam a solução inviável ($\sum w_i x_i > W$) podem ser aceitos com penalização, ou simplesmente descartados. A segunda opção é mais simples para o Knapsack; a primeira é necessária em problemas onde a viabilidade é difícil de manter.

Frequência: $\Phi[i][v]$ conta quantas vezes $x_i = v$ foi visitado. Itens com alta frequência em $x_i = 1$ são candidatos fortes à intensificação; itens raramente explorados indicam oportunidades para diversificação.

6.3 Resultados comparativos

Exemplo

TSP com $n = 30$ cidades (mesma instância do Exercício 3 da Aula 3, coordenadas uniformes em $[0, 100]^2$).

Configuração da BT: $\theta = 6$, tenure adaptativo $\theta \in [5, 8]$, `max_iter` = 2000, frequência de diversificação = 150 iterações sem melhora, 10 soluções elite, inicialização por vizinho mais próximo.

Método	$\overline{f^*}$ (km)	σ_{f^*}	Melhor	NFE médio
HC · 100 reinícios	912	48	841	21 000
SA ($\alpha = 0.95$)	848	21	819	146 000
BT básica	831	14	812	38 000
BT completa	818	8	806	58 000

A BT completa supera todos os métodos em qualidade média e variância, usando 40% das avaliações do SA. A BT básica já supera o SA em qualidade com apenas 26% do orçamento — o mecanismo de memória compensa a ausência de aleatoriedade sendo muito mais eficiente na navegação da paisagem.

Teste de Friedman sobre os quatro métodos: $p < 10^{\{-8\}}$. Post-hoc de Nemenyi: BT completa $\succ$ BT básica $\succ$ SA $\succ$ HC ao nível de 5%. **Wilcoxon** (SA vs. BT completa): $p = 0.0004$, $r = 0.69$ (tamanho de efeito grande).

Diagnóstico da memória de longo prazo: após 2000 iterações, os 5 arcos mais frequentes nas soluções elite apareceram em $> 90\%$ das melhores soluções — indicando que a BT identificou corretamente a "espinha dorsal" do tour ótimo.

7 Parâmetros e Ajuste da Busca Tabu

A Busca Tabu tem menos parâmetros que o SA, mas o tenure θ e as frequências de intensificação/diversificação requerem atenção.

Parâmetro	Papel	Guia de ajuste
Tenure θ	Duração das proibições; controla ciclagem vs. diversidade	$\theta \approx \sqrt{n}$ como ponto de partida; n = dimensão do problema
Tenure adaptativo $[\theta_{\min}, \theta_{\max}]$	Maior robustez que θ fixo	$\theta_{\min} = \lfloor 0.5\sqrt{n} \rfloor$, $\theta_{\max} = \lfloor 1.5\sqrt{n} \rfloor$; ajuste por validação
`freq_div`	Iterações sem melhora antes de diversificar	10–20% de `max_iter`; reduzir se SA supera BT na mesma instância
`freq_intens`	Frequência de retorno às regiões elite	A cada 50–100 iterações; aumentar se qualidade estagna após diversificação
$\lvert\text{Elite}\rvert$	Tamanho do conjunto de soluções elite	5–20; conjuntos maiores diversificam padrões mas custam memória
λ (diversificação)	Peso da penalidade de frequência	Calibrar para que penalidade $\approx$ 10–20% de f típico

Table 1: Guia de parâmetros da Busca Tabu.

Observação

Tenure e tamanho do problema. A regra $\theta \approx \sqrt{n}$ (Glover & Laguna, 1997) é um ponto de partida empírico amplamente validado. Para o TSP com $n = 30$ cidades: $\theta \approx 5.5$, arredondando para 6 — valor que obteve bons resultados no exemplo acima. Para o Knapsack com $n = 20$ itens: $\theta \approx 4.5$, arredondando para 5.

8 Síntese do Módulo 2: Metaheurísticas de Trajetória

Encerramos o Módulo 2 com uma visão integrada dos três algoritmos estudados. A progressão foi deliberada: cada método foi motivado pelas limitações do anterior.

Hill Climbing: ponto de partida conceitual. Eficiente em paisagens unimodais; incapaz de escapar de ótimos locais. A estratégia de reinício adiciona diversificação mas sem aprendizado entre tentativas.

Simulated Annealing: adiciona um mecanismo probabilístico de escape baseado na analogia termodinâmica. O parâmetro temperatura oferece controle contínuo do trade-off exploração/explotação ao longo da execução. Eficaz em problemas contínuos e mistos; requer calibração cuidadosa de T_0, α e L.

Busca Tabu: substitui probabilidade por memória. A lista tabu impede ciclagem de curto prazo; o critério de aspiração garante que proibições não bloqueiem progresso real; a memória de longo prazo permite estratégias globais de intensificação e diversificação. Particularmente eficaz em problemas combinatórios com vizinhanças bem definidas — como o TSP e o Knapsack.

A comparação experimental no TSP ($n = 30$) confirmou a hierarquia BT completa $\succ$ BT básica $\succ$ SA $\succ$ HC. Mas sob a luz do NFL, essa hierarquia vale para **essa classe de problema**: em problemas com paisagens suaves e contínuas, o SA pode superar a BT por não depender de uma definição explícita de vizinhança discreta.

No **Módulo 3**, passamos para algoritmos populacionais: Algoritmos Genéticos, PSO e ACO. A mudança de paradigma — de uma trajetória para uma população — traz novos mecanismos de exploração e novos desafios de projeto.

9 Exercícios

Exercício

1 (Lista Tabu — Ciclagem). Considere o Problema da Mochila com $n = 4$ itens (pesos $= [2, 3, 4, 5]$, valores $= [6, 7, 5, 9]$, $W = 7$) e tenure $\theta = 2$. A partir do estado $\boldsymbol{x} = (0, 0, 0, 0)$, simule manualmente 10 iterações da Busca Tabu **sem** critério de aspiração, escolhendo sempre o melhor vizinho por flip não-tabu. Registre em tabela: iteração, solução corrente, movimento executado, lista tabu e valor de f. Verifique se ocorre ciclagem. Repita com $\theta = 4$ e compare.

Exercício

2 (Critério de Aspiração). Na simulação do Exercício 1 com $\theta = 2$, adicione o critério de aspiração por qualidade ($f(s') < f(s^*)$). Identifique as iterações em que o critério é ativado e mostre um exemplo concreto onde, sem ele, o algoritmo perderia uma solução de melhor valor. O critério de aspiração é ativado com mais frequência com tenure pequeno ou grande? Por quê?

Exercício

3 (Memória de Longo Prazo — Knapsack). Implemente em Python a Busca Tabu básica para o Problema da Mochila com $n = 20$ itens (instância da Aula 1, semente 42). Após 500 iterações, registre a matriz de frequência $\Phi[i][v]$ e responda: (a) Quais 5 itens aparecem com maior frequência em $x_i = 1$? (b) Esses itens coincidem com os selecionados na melhor solução encontrada? (c) Implemente a penalidade de diversificação com $\lambda = 0.3 \cdot \overline{f}$ e compare a qualidade final com e sem diversificação (30 execuções cada).

Exercício

4 (Implementação Completa — BT para TSP). Implemente a Busca Tabu completa (lista tabu FIFO com tenure adaptativo, aspiração por qualidade, intensificação/diversificação por frequência de arcos, conjunto elite de 10 soluções) para o TSP com $n = 30$ cidades (mesma instância do Exercício 4 da Aula 3). Compare com o SA da Aula 3 em 30 execuções independentes cada. Reporte:

(a) Distância média, desvio padrão, melhor resultado e NFE total;

(b) Teste de Wilcoxon com valor-p e tamanho de efeito r;

(c) Curvas de convergência médias com IC 95%;

(d) Os 5 arcos mais frequentes nas soluções elite — eles aparecem na melhor solução encontrada?

Exercício

5 (Tenure Adaptativo). Para a instância do TSP do Exercício 4, compare três configurações: (a) tenure fixo $\theta = 5$; (b) tenure fixo $\theta = 15$; (c) tenure adaptativo $\theta_k \sim \text{Uniforme}\{4, 8\}$. Execute 30 rodadas independentes de cada. Existe diferença estatisticamente significativa entre as três configurações (use Friedman + Nemenyi)? Em qual fase da busca (início, meio ou final) o tenure adaptativo produz maior impacto na taxa de aceitação de movimentos?

Leituras recomendadas:

• Glover, F. Tabu search — Part I. **ORSA Journal on Computing**, 1(3), 190–206, 1989. *(artigo original — leitura obrigatória)*
• Glover, F. Tabu search — Part II. **ORSA Journal on Computing**, 2(1), 4–32, 1990.
• Glover, F., Laguna, M. **Tabu Search**. Kluwer Academic Publishers, 1997. *(referência completa; caps. 1–4 são essenciais)*
• Applegate, D. L. et al. **The Traveling Salesman Problem: A Computational Study**. Princeton University Press, 2006. Caps. 1 e 8.
• Kellerer, H., Pferschy, U., Pisinger, D. **Knapsack Problems**. Springer, 2004. Cap. 1.